

Práctica de Laboratorio 5: Funcionalidad Avanzada de Red en P4

Este laboratorio introduce dos mecanismos de red avanzados implementados en el plano de datos de P4: Notificación Explícita de Congestión (ECN) y segmentación de red basada en VLAN con marcado DSCP por VLAN. Ambos ejercicios utilizan BMv2 (simple_switch) y el mismo flujo de trabajo de compilación y despliegue del laboratorio anterior: compilar con p4c-bm2-ss, iniciar la topología de Mininet con topo.py e instalar las reglas de reenvío usando simple_switch_CLI.

El ejercicio guiado (ECN) proporciona un programa P4 completamente funcional que detecta la congestión de cola en el egreso y la señala modificando los bits ECN de la cabecera IPv4 sin descartar paquetes. La actividad del estudiante (VLAN) requiere completar un esqueleto de programa P4 que implementa etiquetado y desetiquetado IEEE 802.1Q, aislamiento de reenvío basado en VLAN y marcado DSCP por VLAN. Todos los archivos fuente están disponibles en el repositorio del laboratorio.

Una diferencia clave entre los dos ejercicios es el modelo de reenvío: el ejercicio ECN utiliza reenvío L3 con coincidencia de prefijo más largo (LPM) para IPv4 y reescritura de MAC por salto, mientras que el ejercicio VLAN retorna al reenvío L2 y añade manipulación de cabeceras (insertar y eliminar una etiqueta VLAN 802.1Q) como el desafío central de programación.

Ejercicio 1: ECN — Notificación Explícita de Congestión (Guiado)

ECN permite a un dispositivo de red señalar congestión a los extremos sin descartar paquetes. Un host que soporta ECN envía paquetes con el campo ecn de la cabecera IPv4 configurado a 1 o 2 (ECN-Capable Transport, ECT). Cuando la profundidad de la cola de egreso en un conmutador alcanza o supera un umbral definido, el conmutador marca ecn = 3 (Congestion Experienced, CE) en la cabecera del paquete, informando al receptor que la ruta está congestionada. Este ejercicio implementa esa lógica de marcado en P4.

La topología consta de dos conmutadores P4 y cuatro hosts, dos por conmutador. El enlace entre los conmutadores actúa como un cuello de botella

limitado a 0.5 Mbps, lo que permite inducir congestión de forma fiable utilizando un generador de tráfico de fondo.



Figura 1: Topología del ejercicio ECN: dos conmutadores P4 conectados por un enlace de cuello de botella de 0.5 Mbps.

Las asignaciones de hosts y direcciones son:

- s1 (puerto Thrift 9090): h1 en 10.0.1.1/24 en el puerto 1, h11 en 10.0.1.11/24 en el puerto 2, s2 en el puerto 3.
- s2 (puerto Thrift 9091): h2 en 10.0.2.2/24 en el puerto 1, h22 en 10.0.2.22/24 en el puerto 2, s1 en el puerto 3.

Debido a que el ejercicio utiliza reenvío L3, cada host está configurado con una ruta por defecto apuntando a una puerta de enlace virtual. El conmutador reconoce los paquetes destinados a la subred remota y reescribe la MAC de destino Ethernet en cada salto. Esta configuración es aplicada automáticamente por topo.py y no requiere intervención manual.

Paso 1: Compilar el programa P4. Desde la carpeta ecn en el repositorio del laboratorio:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/ecn.p4
```

Además, haga ejecutables los dos scripts auxiliares antes de usarlos:

```
chmod +x send.py receive.py
```

Paso 2: Iniciar la topología de Mininet:

```
sudo python3 mininet/topo.py
```

El script de topología inicia dos conmutadores BMv2 (s1 en el puerto Thrift 9090, s2 en el puerto Thrift 9091), crea cuatro hosts, configura el enlace de cuello de botella y establece la ruta por defecto y las entradas ARP estáticas en cada host para que el tráfico L3 pueda cruzar subredes. Deje esta terminal abierta.

Paso 3: En una segunda terminal, instale las reglas de reenvío en ambos conmutadores:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

Cada archivo de comandos contiene tres entradas LPM: dos entradas /32 para los hosts conectados directamente a ese conmutador, y una entrada /24 para la subred remota reenviada a través del puerto 3 (el enlace entre conmutadores). Las entradas siguen la sintaxis:

```
table_add MyIngress.ipv4_lpm MyIngress.ipv4_forward  
<ip>/<prefijo>=><mac-siguiente-salto><puerto>
```

Paso 4: Verificar la conectividad entre subredes desde la CLI de Mininet:

```
mininet>pingall
```

Debido a que el programa P4 implementa solo reenvío L3 LPM, los hosts en el mismo conmutador y la misma subred no pueden alcanzarse entre sí: las solicitudes ARP son difusiones L2 que la tabla LPM no procesa. Por lo tanto, los pares h1–h11 (ambos en s1, subred 10.0.1.0/24) y h2–h22 (ambos en s2, subred 10.0.2.0/24) siempre fallarán. El resultado esperado es 33 % de pérdida: 8 de 12 pares tienen éxito (todas las combinaciones entre conmutadores) y 4 fallan (los dos pares dentro del mismo conmutador en cada dirección). Este comportamiento es correcto y no indica un error de configuración. Si algún par entre conmutadores también falla, confirme que las reglas de reenvío se instalaron sin errores antes de continuar.

Paso 5: Observar el marcado ECN. Herramientas estándar como ping o iperf no exponen los campos ECN de paquetes individuales ni permiten enviar tráfico con un valor ECN preconfigurado. Dos scripts basados en Scapy abor-

dan esto. send.py construye paquetes UDP con el byte TOS configurado a 1 (ECN = 01, indicando que el emisor soporta ECN) y transmite un paquete por segundo durante un número configurable de envíos. receive.py ejecuta un sniffer Scapy en la interfaz de destino e imprime el byte TOS bruto de cada paquete que llega, haciendo observable directamente la acción de marcado por paquete del conmutador. Abra dos terminales de host adicionales desde la CLI de Mininet:

```
mininet>xterm h2 h22
```

En la terminal de h2, inicie el receptor Scapy que imprime el byte tos bruto para cada paquete UDP que llega:

```
h2# python3 ./receive.py
```

En la terminal de h22, inicie un servidor UDP para absorber el flujo de fondo:

```
h22# iperf -s -u -p 5001
```

Desde la CLI de Mininet, inicie un flujo de fondo de alta tasa desde h11 hacia h22 para saturar el cuello de botella, e inmediatamente después envíe paquetes de prueba marcados con ECN desde h1 hacia h2:

```
mininet>h11 iperf -c 10.0.2.22 -t 15 -u -b 2M -p 5001 &
```

```
mininet>h1 ./send.py 10.0.2.2 "P4 is cool"20
```

Observe el campo tos en la salida de h2. Los paquetes enviados por send.py llevan tos = 1 (ecn = 1, sin señal de congestión). Cuando el flujo de fondo satura la cola del cuello de botella por encima del umbral, el conmutador cambia ecn a 3 (CE), haciendo tos = 3. Después de que el flujo iperf termina y la cola se vacía, el campo debería volver a tos = 1.

Descripción General del Pipeline P4

El programa P4 (p4src/ecn.p4) implementa reenvío LPM IPv4 con marcado ECN en el pipeline de egreso. El elemento de diseño clave es dividir el campo tos de 8 bits de la cabecera IPv4 estándar en dos subcampos accesibles independientemente:

```
header ipv4_t {
    // ... otros campos ...
    bit<6> diffserv; // servicios diferenciados (DSCP)
    bit<2> ecn;      // notificación explícita de congestión
    // ... otros campos ...
}
```

Esta separación permite que el control de egreso modifique los bits ECN sin tocar los bits DSCP. El control de ingreso aplica la tabla LPM, cuya acción `ipv4_forward` establece el puerto de egreso y reescribe ambas direcciones Ethernet en cada salto:

```
action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {
    standard_metadata.egress_spec = port;
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
    hdr.ethernet.dstAddr = dstAddr;
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
}
```

El control de egreso lee `standard_metadata.enq_qdepth`, la profundidad de la cola en el momento en que el paquete fue encolado. Si esa profundidad alcanza o supera `ECN_THRESHOLD = 10` y el paquete es capaz de ECN (`ecn == 1` o `ecn == 2`), el conmutador llama a `mark_ecn()`, que establece `hdr.ipv4.ecn = 3`:

```
const bit<19>ECN_THRESHOLD = 10;

// En MyEgress.apply():
if (hdr.ipv4.ecn == 1 || hdr.ipv4.ecn == 2) {
    if (standard_metadata.enq_qdepth >= ECN_THRESHOLD) {
        mark_ecn(); // establece hdr.ipv4.ecn = 3
    }
}
```

El marcado ocurre en egreso — no en ingreso — porque `enq_qdepth` solo se llena después de que el paquete ha sido colocado en la cola de salida, lo que ocurre entre los pipelines de ingreso y egreso. Por lo tanto, el valor no está

disponible para el control de ingreso. Debido a que el campo ecn cambia, My-ComputeChecksum debe recalcular el checksum IPv4 listando diffserv y ecn como campos separados, ya que ahora ocupan el byte que antes se llamaba tos. El desanizador emite la cabecera Ethernet seguida de la cabecera IPv4.

Actividad 1: Ejercicio 4 — Segmentación VLAN y Marcado DSCP

Esta actividad implementa el etiquetado VLAN IEEE 802.1Q en el plano de datos de P4. Dos conmutadores P4 dividen cuatro hosts en dos VLAN. Los paquetes provenientes de los hosts llegan sin etiquetar al conmutador de acceso; el conmutador inserta una cabecera VLAN 802.1Q y reenvía el paquete a través del enlace troncal al conmutador par. El conmutador receptor inspecciona la cabecera VLAN, reenvía el paquete al puerto del host correcto y elimina la cabecera VLAN antes de la entrega. Los paquetes cuya combinación de MAC destino e ID de VLAN no tiene una entrada coincidente se descartan silenciosamente en el plano de datos, imponiendo el aislamiento de VLAN sin intervención del controlador.

Adicionalmente, el programa P4 escribe un valor DSCP en la cabecera IPv4 de acuerdo con la VLAN entrante. Esto permite que dispositivos posteriores o herramientas de monitoreo identifiquen la clase de tráfico usando el campo DSCP incluso después de que la etiqueta VLAN haya sido eliminada en el conmutador de destino.

La topología consta de dos conmutadores P4 y cuatro hosts:

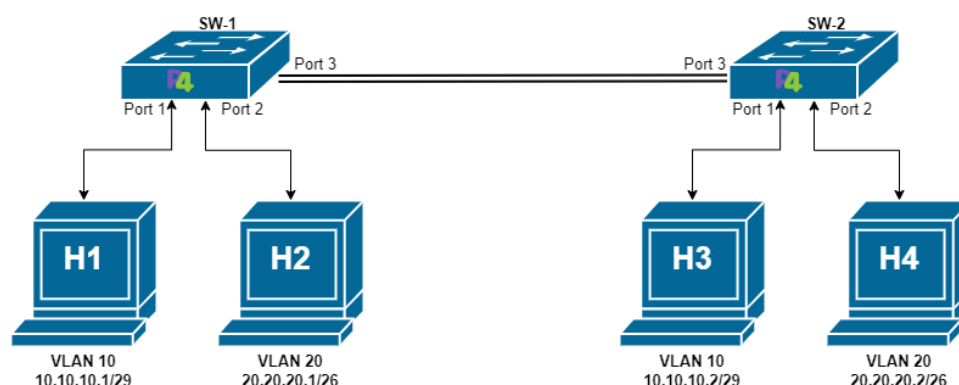


Figura 2: Topología del ejercicio VLAN: dos conmutadores de acceso conectados por un enlace troncal.

Las asignaciones de hosts y direcciones son:

- s1 (puerto Thrift 9090): h1 en 10.10.10.1/29 (VLAN 10) en el puerto 1, h2 en 20.20.20.1/26 (VLAN 20) en el puerto 2, s2 en el puerto 3 (truncal).
- s2 (puerto Thrift 9091): h3 en 10.10.10.2/29 (VLAN 10) en el puerto 1, h4 en 20.20.20.2/26 (VLAN 20) en el puerto 2, s1 en el puerto 3 (truncal).

Descripción General del Pipeline P4

El programa P4 (p4src/main.p4) implementa el procesamiento bidireccional de VLAN. La cabecera 802.1Q se define junto con las cabeceras Ethernet e IPv4 estándar:

```
header vlan_802_1q_t {
  bit<3>  pri;          // código de punto de prioridad (PCP)
  bit<1>  cfi;          // indicador de descarte elegible (DEI)
  bit<12> vid;          // identificador de VLAN
  bit<16> ether_type;  // protocolo encapsulado
}
```

Se definen dos identificadores de VLAN y dos valores DSCP como constantes: VLAN_10 = 0x00a (decimal 10), VLAN_20 = 0x014 (decimal 20), DSCP_VLAN_10 = 10 (AF11) y DSCP_VLAN_20 = 20 (AF22). El analizador se ramifica según el ether_type de Ethernet: los paquetes con 0x0800 (IPv4) van directamente a parse_ipv4; los paquetes con 0x8100 (TPID VLAN) extraen primero la cabecera VLAN y luego continúan a parse_ipv4.

El control de ingreso IngressPipeImpl contiene dos tablas principales:

- set_vlan_tag: se compara con la dirección MAC origen cuando el ether_type entrante es 0x0800 (tráfico de host sin etiquetar). La acción add_vlan_tag valida la cabecera VLAN, asigna el ID de VLAN correcto, marca el campo DSCP de la cabecera IPv4, mueve el ether_type original a la cabecera VLAN, establece hdr.ethernet.ether_type = 0x8100 y reenvía el paquete al puerto 3 (truncal).

- `extract_vlan_tag`: se compara con la dirección MAC destino y el ID de VLAN cuando el `ether_type` entrante es 0x8100 (tráfico troncal etiquetado). La acción `set_egress_port` selecciona el puerto de host correcto. Después de que la tabla se aplica, `hdr.ethernet.ether_type` se restaura desde la cabecera VLAN y `hdr.vlan_802_1q.setInvalid()` elimina la etiqueta antes de la entrega. La acción predeterminada `@defaultonly drop` impone el aislamiento de VLAN: cualquier paquete cuya combinación de MAC destino e ID de VLAN no tenga entrada se descarta silenciosamente.

Debido a que el campo DSCP de la cabecera IPv4 se escribe durante el etiquetado, `ComputeChecksumImpl` debe recalcular el checksum de la cabecera IPv4 después de cualquier modificación. El desanalizador emite la cabecera Ethernet, luego la cabecera VLAN (solo cuando es válida), y luego la cabecera IPv4.

Secciones TODO para Completar

Abra `p4src/main.p4` de la carpeta VLAN en el repositorio del laboratorio y complete todas las secciones marcadas con TODO. Los cambios requeridos abarcan las siguientes áreas:

- **Constantes de ID de VLAN**: abra `s1-commands.txt` y localice las entradas de `extract_vlan_tag`. Cada entrada incluye un ID de VLAN decimal como la segunda clave de coincidencia. Convierta esos valores decimales a hexadecimal de 12 bits y asígneles a `VLAN_10` y `VLAN_20`. Los archivos de comandos son fijos; las constantes que asigne deben coincidir con las claves en esos archivos, por lo que solo hay un par correcto de valores.
- **Constantes DSCP**: defina dos valores de 6 bits (rango 0–63) para asignar una clase de tráfico a cada VLAN. Los valores esperados son `DSCP_VLAN_10 = 10 (AF11)` y `DSCP_VLAN_20 = 20 (AF22)`. Estas constantes determinan el byte TOS visible en las capturas de paquetes en los hosts de destino.

- **Analizador (parse_vlan):** añada una instrucción `transition parse_ipv4` después de `packet.extract(hdr.vlan_802_1q)`, para que el procesamiento IPv4 continúe correctamente cuando haya una cabecera VLAN presente.
- **Acción add_vlan_tag — Marcado DSCP:** después de asignar el ID de VLAN, verifique `hdr.ipv4.isValid()` y escriba la constante DSCP apropiada en los bits DSCP de la cabecera IPv4 según el puerto de ingreso.
- **Bloque apply — Eliminación de VLAN:** después de que `extract_vlan_tag` haya coincidido y se haya restaurado `hdr.ethernet.ether_type`, llame a `hdr.vlan_802_1q.setInvalid()` para que el desanalizador no emita la cabecera VLAN hacia el host de destino.
- **Checksum:** añada una llamada `update_checksum` en `ComputeChecksumImpl`, protegida por `hdr.ipv4.isValid()`, que cubra todos los campos de la cabecera IPv4 y almacene el resultado en `hdr.ipv4.hdr_checksum`.

Compilar y Desplegar

Una vez completada la implementación, compile el programa desde la carpeta VLAN:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

Inicie la topología de Mininet:

```
sudo python3 mininet/topo.py
```

La topología utiliza `net.staticArp()` para precargar las tablas ARP de todos los hosts después de `net.start()`, eliminando la resolución ARP como una variable en el experimento VLAN. En una segunda terminal, instale las reglas de reenvío:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

Cada archivo de comandos contiene entradas para `set_vlan_tag` (una por cada host conectado a ese conmutador, todas dirigiendo el tráfico al puerto 3) y entradas para `extract_vlan_tag` (una por host, con clave en MAC destino e ID de VLAN en decimal, dirigiendo el tráfico al puerto de host correspondiente). Las entradas de `extract_vlan_tag` siguen el formato:

Verificar el Aislamiento VLAN

Desde la CLI de Mininet, ejecute una prueba de conectividad completa:

```
mininet>pingall
```

Observe qué pares tienen éxito y cuáles fallan, y relacione el resultado con la pertenencia a VLAN de cada host. Luego pruebe la conectividad intra-VLAN y entre VLAN explícitamente:

```
mininet>h1 ping h3 -c 3
```

```
mininet>h2 ping h4 -c 3
```

```
mininet>h1 ping h4 -c 3
```

```
mininet>h2 ping h3 -c 3
```

Los hosts h1–h3 (VLAN 10) y h2–h4 (VLAN 20) deben comunicarse exitosamente. Los pares h1–h4 y h2–h3 no deben intercambiar ningún paquete. La caída entre VLAN ocurre completamente en el plano de datos P4 a través de la acción predeterminada `@defaultonly drop` de `extract_vlan_tag`: ningún paquete llega a un host no deseado.

Verificar el Marcado DSCP

Abra sesiones xterm para los hosts de destino:

```
mininet>xterm h3 h4
```

En la terminal de h3, capture el tráfico entrante con verbosidad para exponer la cabecera IP:

```
h3# tcpdump -i eth0 -v -c 10
```

En la terminal de h4, haga lo mismo:

```
h4# tcpdump -i eth0 -v -c 10
```

Desde la CLI de Mininet, genere tráfico de prueba mientras las capturas están ejecutándose:

```
mininet>h1 ping h3 -c 5
```

```
mininet>h2 ping h4 -c 5
```

Inspeccione el campo tos reportado por tcpdump. El valor DSCP ocupa los 6 bits superiores del byte TOS (desplazados a la izquierda por 2), con los dos bits ECN puestos a cero. Los valores TOS esperados son: DSCP_VLAN_10 = 10 $\Rightarrow$ tos = 40 (0x28), y DSCP_VLAN_20 = 20 $\Rightarrow$ tos = 80 (0x50). Verifique que cada captura muestre el valor que corresponde a la VLAN del emisor.

Tenga en cuenta que la etiqueta VLAN no es visible en la salida de tcpdump en el host: el conmutador la elimina antes de la entrega. El marcado DSCP en la cabecera IPv4 es el único campo por VLAN que sobrevive de extremo a extremo, porque es parte de la cabecera IP en lugar de una cabecera de capa de enlace.

Preguntas de Discusión

1. ¿Qué campos de la cabecera IPv4 se modifican en el pipeline ECN y cuáles en el pipeline VLAN? En ambos casos se debe recalcular el checksum IPv4 aunque el objetivo del reenvío sea diferente. Explique por qué se requiere el recálculo del checksum en cada caso.
2. El marcado ECN ocurre en el pipeline de egreso, mientras que el marcado DSCP en el ejercicio VLAN ocurre en el pipeline de ingreso. Identifique la razón técnica que impide que el marcado ECN se realice en ingreso, y explique por qué el marcado DSCP no tiene esa restricción.
3. ¿Cuál es la diferencia fundamental entre un campo que es local a un único segmento de enlace, como una etiqueta VLAN 802.1Q, y un campo que se conserva de extremo a extremo, como DSCP o ECN? ¿Cómo explica esta distinción el comportamiento del conmutador en

los puertos de acceso (inserción y eliminación de etiquetas) frente al comportamiento en el puerto troncal (preservación de la etiqueta)?